Experiment No. 10

Write the Python program for Map Coloring to implement CSP

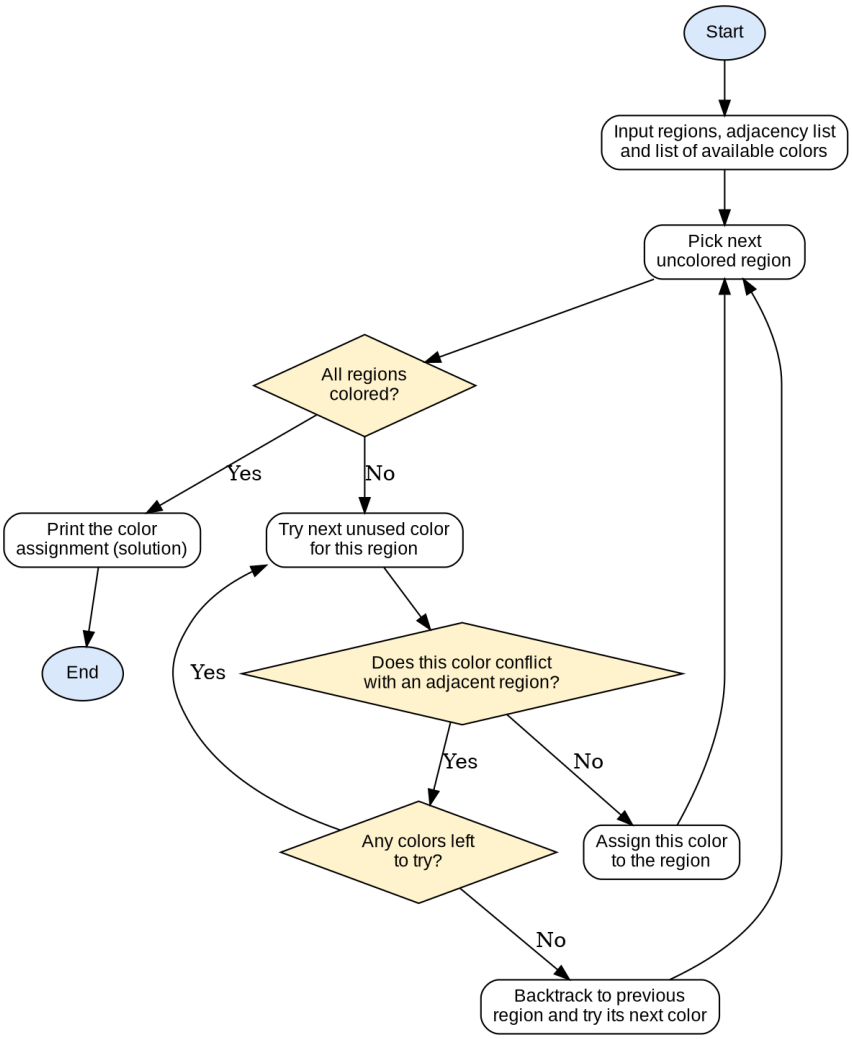
Aim

To write a Python program that solves the Map Coloring problem as a Constraint Satisfaction Problem (CSP) using backtracking, such that no two adjacent regions share the same color.

Algorithm

1. Start.
2. Represent the map as a graph in which regions are nodes and shared borders between regions are edges (adjacency list).
3. Define the list of available colors.
4. Select the next uncolored region.
5. Try assigning the next available color to the region such that the constraint is satisfied, i.e. no adjacent region already has the same color.
6. If a valid color is found, assign it to the region and move on to the next uncolored region.
7. If no valid color can be assigned, backtrack to the previous region and try its next available color.
8. Repeat steps 4-7 until all regions are colored (a solution is found) or all possibilities are exhausted (no solution exists).
9. Display the color assigned to each region.
10. Stop.

Flowchart



Program

```
def is_valid(region, color, coloring, adjacency):
    for neighbor in adjacency[region]:
        if coloring.get(neighbor) == color:
            return False
    return True


def backtrack(regions, colors, adjacency, coloring,
index=0):
    if index == len(regions):
        return True

    region = regions[index]
```

```
    for color in colors:
        if is_valid(region, color, coloring, adjacency):
            coloring[region] = color
            if backtrack(regions, colors, adjacency,
coloring, index + 1):
                return True
            del coloring[region]

    return False


adjacency = {
    'WA': ['NT', 'SA'],
    'NT': ['WA', 'SA', 'Q'],
    'SA': ['WA', 'NT', 'Q', 'NSW', 'V'],
    'Q': ['NT', 'SA', 'NSW'],
    'NSW': ['SA', 'Q', 'V'],
    'V': ['SA', 'NSW']
}

regions = list(adjacency.keys())
colors = ['Red', 'Green', 'Blue']
coloring = {}

if backtrack(regions, colors, adjacency, coloring):
    print("Solution found:")
    for region in regions:
        print(f"{region}: {coloring[region]}")
else:
    print("No solution exists")
```

Result

The program was executed successfully and the output was verified as correct.

```
Solution found:
WA: Red
NT: Green
SA: Blue
Q: Red
NSW: Green
```

```
V: Red
```

Sample Output — Running in VS Code

Screenshot of the program being run in the Visual Studio Code integrated terminal: